“SMART TAX MANAGEMENT SYSTEM”

A

Project Report

submitted

in partial fulfillment

for the award of the Degree of

Bachelor of Technology

in Department of Computer Science and Engineering

Project Id:
SKIT/2025-26/AI/17

Project Mentor:
Dr. Mehul Maharishi
Head of Department

Submitted By:
Khushal Jangid (22ES-KCA057)

Gaurav Pratap Singh (22ES-KCA035)

Kapil Jarwal (22ESKCA054)

Himanshu Meena (22ES-KCA047)

Department of Computer Science and Engineering
Swami Keshvanand Institute of Technology, M & G, Jaipur
Rajasthan Technical University, Kota
Session 2025-2026

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that Mr. Khushal Jangid, a student of B.Tech (Computer Science & Engineering (AI)) 8 semester has submitted his/her Project Report entitled ”SMART TAX MANAGEMENT SYSTEM” under my guidance

Mentor	**Coordinator**
Dr. Mehul Maharishi	Mr. Manish Bharadwaj
Head of Department	Assistant Professor
Signature.....	Signature.....

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that Mr. Gaurav Pratap Singh, a student of B.Tech (Computer Science & Engineering (AI)) 8 semester has submitted his/her Project Report entitled ”SMART TAX MANAGEMENT SYSTEM” under my guidance

Mentor	**Coordinator**
Dr. Mehul Maharishi	Mr. Manish Bharadwaj
Head of Department	Assistant Professor2
Signature.....	Signature.....

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that Mr. Kapil Jarwal, a student of B.Tech (Computer Science & Engineering (AI)) 8 semester has submitted his/her Project Report entitled ”SMART TAX MANAGEMENT SYSTEM” under my guidance

Mentor	**Coordinator**
Dr. Mehul Maharishi	Mr. Manish Bharadwaj
Head of Department	Assistant Professor2
Signature.....	Signature.....

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that Mr. Himanshu Meena, a student of B.Tech (Computer Science & Engineering (AI)) 8 semester has submitted his/her Project Report entitled ”SMART TAX MANAGEMENT SYSTEM” under my guidance

Mentor	**Coordinator**
Dr. Mehul Maharishi	Mr. Manish Bharadwaj
Head of Department	Assistant Professor2
Signature.....	Signature.....

DECLARATION

We declare that the project report titled **"Smart Property Tax Management System (SMPTMS)"** is an original record of the work we conducted at Swami Keshvanand Institute of Technology, Management and Gramothan, Jaipur. This project was completed under the guidance of **Dr. Mehul Maharishi** and the coordination of **Mr. Manish Bharadwaj** within the Department of Computer Science and Engineering (AI). We are submitting this report in partial fulfillment of the requirements for the Bachelor of Technology (B.Tech) degree. To our knowledge, this work has not been submitted for any other degree or program at any other institution.

Team Members **Signature**

Khushal Jangid (22ESKCA057)

Gaurav Pratap Singh (22ESKCA035)

Kapil Jarwal (22ESKCA054)

Himanshu Meena (22ESKCA047)

Acknowledgement

We thank our mentor, **Dr. Mehul Maharishi**, for his guidance and help during this project. His suggestions helped us solve many problems we faced during the work. We also thank our project coordinator, Mr. Manish Bharadwaj, for his continuous help and useful advice from beginning to end.

We also thank the faculty members of the Department of Computer Science (AI) at SKIT, Jaipur, for providing a good learning environment. We are thankful to the college management for giving us access to the resources and facilities needed to complete this project.

Lastly, we thank our classmates and everyone who supported us in any way, directly or indirectly, in completing this project..

Team Members:
Khushal Jangid (22ESKCA057)
Gaurav Pratap Singh (22ESKCA035)
Kapil Jarwal (22ESKCA054)
Himanshu Meena (22ESKCA047)

Abstract

Property tax collection in most Indian municipalities remains a manual and inefficient process, characterized by numerous administrative loopholes. Many smaller urban local bodies continue to rely on outdated assessment methods and property data that lack accuracy. For citizens, the lack of transparency in tax calculation and the cumbersome payment process present significant hurdles. Conversely, municipal staff allocate a disproportionate amount of time to revenue collection and enforcement, detracting from the delivery of essential public services.

The Smart Property Tax Management System (SMPTMS) addresses these challenges, specifically for Jaipur Nagar Nigam. We have developed a comprehensive web-based application that automates the entire tax cycle: property registration, tax assessment, online payments, and receipt generation. The platform allows citizens to register their properties, view their calculated tax based on plot area, zone, and land use, and complete payments through a secure gateway—eliminating the need for manual intervention and frequent visits to municipal offices.

The system features a dashboard for municipal officials, incorporating a live map (developed using Leaflet.js) that displays all registered properties in Jaipur, categorized by payment status. The administrative interface provides advanced analytics, tools for identifying tax defaulters, and reporting features designed to optimize revenue management.

Developed using Next.js 14, TypeScript, Supabase (PostgreSQL), Tailwind CSS, and Razorpay, the SMPTMS is a cost-effective and transparent solution for municipal tax management. The calculation methodology—Tax = (Plot Area x DLC Rate)/2000—ensures simplicity and transparency. The implementation of SMPTMS demonstrates the potential for digital transformation in municipal tax administration, even in medium-sized cities like Jaipur.

Table of Content

1 Introduction **2**

1.1 Problem Statement and Objective 2

1.2 Literature Survey / Market Survey 2

1.3 Introduction to Project 3

1.4 Proposed Logic / Algorithm / Business Plan / Solution 4

1.5 Scope of the Project 4

2 Software Requirement Specification **6**

2.1 Overall Description 6

2.1.1 Product Perspective 6

2.1.1.1 System Interfaces 6

2.1.1.2 User Interfaces 7

2.1.1.3 Hardware Interfaces 7

2.1.1.4 Software Interfaces 7

2.1.1.5 Functional Requirement 8

2.1.1.6 User Types 8

2.1.1.7 Constraints 8

2.1.1.8 Assumptions and Dependencies 9

3 System Design Specification **10**

3.1 System Architecture 10

3.2 Module Decomposition Description 10

3.3 High Level Design Diagrams 12

3.3.1 Use Case Diagram 12

3.3.2 Activity Diagram 13

3.3.3 Data-Flow Diagram 14

3.3.4 Class Diagram 15

3.3.5 Sequence Diagram 16

4 Methodology and Team **17**

4.1 Introduction to Agile Methodology 17

4.2 Team Members, Roles & Responsibilities 19

5 System Testing **22**

5.1 Introduction 22

5.2 Types of Testing Performed 22

5.2.1 Unit Testing 22

5.2.2 Integration Testing 22

5.2.3 System Testing 23

5.3 Test Cases 24

6 Project Screenshots **28**

6.1 Introduction 28

6.2 Public Landing Page 28

6.3 Citizen Interface 30

6.4 Admin Control Center 35

6.5 GIS City Map 37

7 Conclusion and Future Scope **38**

7.1 Conclusion 38

7.2 Future Scope 38

8 UN Sustainable Development Goals **40**

8.1 Mapping with UN Sustainable Development Goals 40

References **40**

Project Poster **43**

Certificate **44**

GitHub Link **45**

List of Figures

3.1 Use Case Diagram of SMPTMS 12
3.2 Activity Diagram – Citizen Tax Payment Flow 13
3.3 Data-Flow Diagram of SMPTMS 14
3.4 Class Diagram of SMPTMS 15
3.5 Sequence Diagram – Payment Processing Flow 16

6.1 SMPTMS Homepage – Simple Property Tax, Transparent Management 28
6.2 Landing Page – How It Works Section (Register, Calculate, Pay, Receipt) 29
6.3 Citizen Login / Registration Page – Secure Auth Portal 30
6.4 Citizen Dashboard – Property Portfolio, Dues, and Quick Actions . 31
6.5 Property Registration – Step 1: Location & Ownership Details with Map Pin 31
6.6 Property Registration – Step 2: Dimensions & Estimated Annual Tax Calculation 32
6.7 Tax Payments Page – Select Property to Pay Tax (Approved Asset View) 32
6.8 Bill Summary – DLC Rate Breakdown, Rebate Calculation, Final Tax Payable 33
6.9 Razorpay Payment Gateway – Cards, Netbanking, UPI, Wallet Options 33
6.10 Payment Success Confirmation – Transaction ID, Amount, Print Receipt Button 34
6.11 Official Property Tax Receipt – Jaipur Nagar Nigam, Razorpay Transaction Details 34
6.12 Admin Dashboard Overview – Total Tax Collected, Properties, Pending Approvals, Defaulters 35

6.13 Admin – All Properties Database with Approve/Reject Actions . . . 35
6.14 Admin Analytics – Zone-wise Bar Chart, Property Status Pie Chart, Monthly Collection Trend 36
6.15 Admin Tax Rate Configuration – DLC Rates, Zone Factors, Live Tax Preview 36
6.16 Jaipur Property Tax GIS Map – 21 Properties with Paid/Pending/Defaulter Markers and Ward Boundaries 37

List of Tables

2.1 Functional Requirements of SMPTMS 8

4.1 Member-1 Roles and Responsibilities 19
4.2 Member-2 Roles and Responsibilities 20
4.3 Member-3 Roles and Responsibilities 20
4.4 Member-4 Roles and Responsibilities 21

5.1 Test Cases – User Management Module 24
5.2 Test Cases – Tax Calculation Module 25
5.3 Test Cases – Payment and Receipt Module 26
5.4 Test Cases – GIS Map and Surveying Module 27

8.1 SDG 11 Mapping for SMPTMS 40
8.2 Project Technical Summary 45

Chapter 1
Introduction

1.1 Problem Statement and Objective

The property tax system in Jaipur currently faces significant challenges, primarily due to its reliance on outdated manual processes. A large number of properties remain unregistered as a result of the lack of a streamlined digital registration mechanism. Tax assessments are often inconsistent, influenced by antiquated records or subjective administrative decisions. The lack of transparency in tax calculation methods has fostered public distrust. Furthermore, municipal personnel spend excessive time on revenue collection and enforcement, diverting resources from the improvement of public services.

Our project aims to address these issues by implementing a digital platform that facilitates property registration, tax assessment, online payments, and receipt generation. This centralized system provides a transparent and efficient interface for both citizens and municipal staff, reducing administrative overhead and eliminating the need for frequent office visits.

India's property tax collection is approximately 0.2% of GDP, which is significantly lower than in developed economies such as the United States or Canada, where property tax revenue can account for 3–4% of GDP. This shortfall impacts the development of basic infrastructure, as Urban Local Bodies (ULBs) depend on these revenues to fund essential services such as roads, sanitation, and water supply.

1.2 Literature Survey / Market Survey

Larger Indian metropolitan areas, such as Pune (PCMC) and Bangalore (BBMP), have demonstrated significant success with digital property tax systems, enhancing tax compliance and reducing administrative burdens. However, these platforms are

often complex, costly, and specifically designed for major urban centers. Consequently, there is a pressing need for a more streamlined and cost-effective solution tailored to the requirements of cities like Jaipur, which may have limited administrative resources.

International experience demonstrates that the integration of tax management systems with digital mapping provides tangible benefits. For instance, the Valuation Office Agency in the UK and county-level GIS systems in the US facilitate efficient property discovery, accurate valuation, and data-driven planning. The SMPTMS incorporates key features from these established models, adapted to the specific needs of the Jaipur municipality.

Academic research has consistently identified three primary issues in India's property tax administration: outdated or unmapped records, inconsistent tax calculation methodologies, and a lack of public trust, which adversely affects voluntary compliance.

1.3 Introduction to Project

The SMPTMS provides citizens with a streamlined dashboard for property registration, enabling them to input essential details such as plot number, area, zone, and land use category. The system then automatically calculates the due tax based on the applicable District Level Committee (DLC) rates. Upon confirmation of the assessed tax, users can complete the payment securely through the integrated Razorpay gateway and download an official digital receipt, eliminating the need for in-person administrative visits.

Administrators are provided with a dedicated control panel featuring a live GIS map, which visualizes all registered properties in Jaipur through color-coded status indicators. The administrative interface enables officials to identify tax defaulters, manage property registrations, update DLC rates by zone, and generate comprehensive reports filtered by ward or zone. The SMPTMS is deployed and accessible at smptms.vercel.app, utilizing a Supabase PostgreSQL backend integrated with authentic municipal data from Jaipur. The system offers a robust solution to the

complexities of traditional property tax administration, emphasizing transparency, efficiency, and data-driven decision-making.

1.4 Proposed Logic / Algorithm / Business Plan / Solution

The solution is structured around a straightforward end-to-end workflow. A property owner registers on the platform and submits property details including plot number, area in square metres, zone category, and land use type (residential, commercial, or industrial). The backend calculates annual tax automatically using the formula:

$$\textbf{Annual Tax} = (\textbf{Plot Area} \times \textbf{DLC Rate}) / 2000$$

The DLC rate is configured per zone by the admin and reflects the government's published District Level Circle rates for Jaipur. Once the tax amount is displayed, the citizen proceeds to payment through Razorpay's hosted checkout. On successful payment, Razorpay sends a webhook callback to the Next.js API server, which verifies the payment signature cryptographically before updating the property status in Supabase and generating a downloadable PDF receipt.

On the administrative side, all registered properties are fetched from Supabase and rendered as markers on a Leaflet.js map centred on Jaipur coordinates. Green markers represent paid properties and orange markers represent pending ones. Admins can filter by ward, zone, or payment status, click any marker to view full property details, and export zone-wise tax reports for departmental use.

The three-agent architecture separates the authentication module, tax calculation engine, and payment processing module into independent, maintainable units — making the system robust and easy to extend.

1.5 Scope of the Project

SMPTMS covers property tax administration specifically for Jaipur Nagar Nigam. The system's functional scope includes property registration, automated DLC-rate-based tax calculation, online payment processing via Razorpay, digital receipt generation, GIS-based property mapping with Leaflet.js, and administrative reporting

with zone-wise and ward-wise filters.

The current version does not cover other municipal revenue streams such as trade licenses, water bills, or building plan approvals. It is not intended to replace state-level platforms like Rajasthan's ULBHRMS but to function as a lightweight, accessible front-end layer that any ward office can operate without requiring specialized training or infrastructure investment.

The platform is designed specifically for the Jaipur context, using real zone coordinates and DLC rates applicable to Jaipur Nagar Nigam's jurisdiction. Future iterations could extend the system to other Rajasthan municipalities with minimal configuration changes.

Chapter 2
Software Requirement Specification

2.1 Overall Description

This Software Requirement Specification (SRS) lays out exactly what the SMPTMS system—developed for Jaipur Nagar Nigam—needs to do, covering both what it must accomplish (the functional bits) and how it should perform (the non-functional stuff). Here, you'll get a sense of the bigger picture: how the system fits in, who's going to use it, any rules or constraints we have to work under, and the main assumptions behind how everything's supposed to work. SMPTMS is a web application that handles property tax management and controls access via different user roles. It uses a three-layer setup: the Next.js frontend, business logic through Next.js API routes, and a PostgreSQL database powered by Supabase.

2.1.1 Product Perspective

2.1.1.1 System Interfaces

SMPTMS connects with three outside systems. Supabase is the main backend—it manages all the data storage, live updates, and user authentication, all through its PostgreSQL database. The application talks to Supabase using REST APIs over HTTPS to keep things secure. For handling payments, everything goes through Razorpay. It supports both sandbox mode for testing and production mode for real payments. The GIS dashboard—basically, the map interface—gets its base layer from Google Maps tile servers, and Leaflet.js renders the final map view users see. Data is sent around as JSON, always over secure HTTPS, so everything stays compatible and safe no matter where you access it.

2.1.1.2 User Interfaces

There are two main user journeys here, split by role. Citizens log in to find a personal dashboard where they can see all their properties, check if they owe any taxes, dig through payment history, or register a new property. This dashboard is designed to work smoothly whether you're on your phone or your computer. On the flip side, admins get an entirely separate panel. They see a live map of every property in Jaipur, with tools to drill down by zone, view who hasn't paid, configure different property rates, or export reports. Navigation happens via a sidebar with simple labels, so admin users don't need to be tech experts.

2.1.1.3 Hardware Interfaces

The whole SMPTMS system lives in the cloud, so nobody needs fancy or proprietary hardware to use it. Any device—a smartphone, tablet, or regular computer—as long as it runs a modern web browser and has an internet connection, can use everything the system offers. The application is hosted on Vercel's edge network, which means users across India get quick and reliable access wherever they are.

2.1.1.4 Software Interfaces

The software interfaces used in SMPTMS are as follows:

- **Frontend Framework:** Next.js 14 (App Router) with TypeScript
- **Styling:** Tailwind CSS v3
- **Backend / Database:** Supabase (PostgreSQL + Supabase Auth + Supabase Storage)
- **GIS / Mapping:** Leaflet.js v1.9.4 with react-leaflet v4.2.1
- **Payment Gateway:** Razorpay SDK (Node.js)
- **Deployment Platform:** Vercel
- **Runtime Environment:** Node.js v18+

2.1.1.5 Functional Requirement

The functional requirements of SMPTMS are defined as follows:

ID	Requirement
FR-001	Secure user registration and login with role-based access control (Citizen / Admin) via Supabase Auth
FR-002	Citizen dashboard displaying registered property portfolio, outstanding dues, and payment history
FR-003	CRUD operations for property records including plot number, area, zone, and land use type
FR-004	Automated tax calculation using the formula: Tax = (Plot Area x DLC Rate) / 2000
FR-005	Online payment integration via Razorpay with webhook-based payment verification
FR-006	Downloadable PDF receipt generation after successful payment
FR-007	Admin GIS map showing all properties with colour-coded payment status markers
FR-008	Admin defaulter list with ward-wise and zone-wise filtering
FR-009	Zone-wise DLC rate configuration by admin
FR-010	Exportable zone-wise and ward-wise tax collection reports

Table 2.1: Functional Requirements of SMPTMS

2.1.1.6 User Types

There are two user groups on this platform. Citizens are property owners, ranging from everyday folks who use smartphones casually to business owners dealing with lots of property. They need things straightforward—clear steps, minimal jargon, and unambiguous confirmation when payments go through. Admins are the municipal staff working out of Jaipur Nagar Nigam ward offices. They're generally comfortable with computers, but aren't tech experts. Their tools have to be clear, with easy-to-read controls, visual status cues, and quick access to reports—no database know-how required.

2.1.1.7 Constraints

This system needs to work on free or budget-friendly infrastructure because municipal projects don't get much IT funding. Internet speed at ward offices isn't always reliable, so the frontend is tuned for fast loading even on slow connections, using Next.js server-side rendering and static generation. Every financial transac-

tion must comply with Razorpay integration rules and PCI-DSS security standards.

2.1.1.8 Assumptions and Dependencies

We're assuming ward offices have at least basic computers and stable internet. For accurate tax calculations, admins have to input and keep DLC rates up to date. The platform relies on Supabase for authentication, Razorpay for payments, and Vercel for hosting. Property owners must provide valid IDs and mobile numbers for payment verification via Razorpay.

Chapter 3
System Design Specification

3.1 System Architecture

SMPTMS follows a three-tier client-server architecture. The presentation layer is built on Next.js 14 with the App Router, running in the user's browser as a React-based single-page application. The application layer handles all business logic — including tax calculation, authentication middleware, payment processing, and webhook verification — through Next.js API routes and server components executing in a Node.js runtime environment. The data layer resides on Supabase, which manages a PostgreSQL relational database, file storage for receipt PDFs, and real-time data subscriptions.

All three tiers communicate exclusively over HTTPS. Citizens and administrators access the same Vercel deployment at smptms.vercel.app but are presented with entirely different interfaces based on their assigned role, which is verified on every request through Supabase Auth middleware running at the edge.

The payment flow operates through a dedicated sub-process: when a citizen initiates payment, the Next.js API creates a Razorpay order object, the citizen completes payment on Razorpay's hosted checkout page, and Razorpay sends a signed webhook callback to the server. The server verifies the cryptographic signature before updating the property payment status in Supabase and generating the receipt document.

3.2 Module Decomposition Description

SMPTMS is organized into four primary modules, each owned and developed by a dedicated team member:

Module 1 — User Management (Khushal Jangid): Handles user authentication, role-based access control (Citizen/Admin), session management via Supabase Auth, and the citizen-facing dashboard displaying property portfolio and dues.

Module 2 — Tax Calculation (Gaurav Pratap Singh): Covers the Supabase schema design for property records and tax assessments, DLC rate configuration per zone, automated tax formula implementation (Tax = Plot Area x DLC Rate / 2000), and the assessment approval workflow for administrators.

Module 3 — Testing Module (Kapil Jarwal): Encompasses the complete quality assurance pipeline including Postman API testing, integration testing across all modules, automated test case development, Vercel deployment management, and bug tracking documentation.

Module 4 — Surveying / GIS (Himanshu Meena): Manages the Leaflet.js GIS map integration with real Jaipur coordinates, colour-coded property marker rendering, ward-wise and zone-wise filtering, Google Maps satellite tile toggle, and survey data upload with version history tracking.

3.3 High Level Design Diagrams

3.3.1 Use Case Diagram

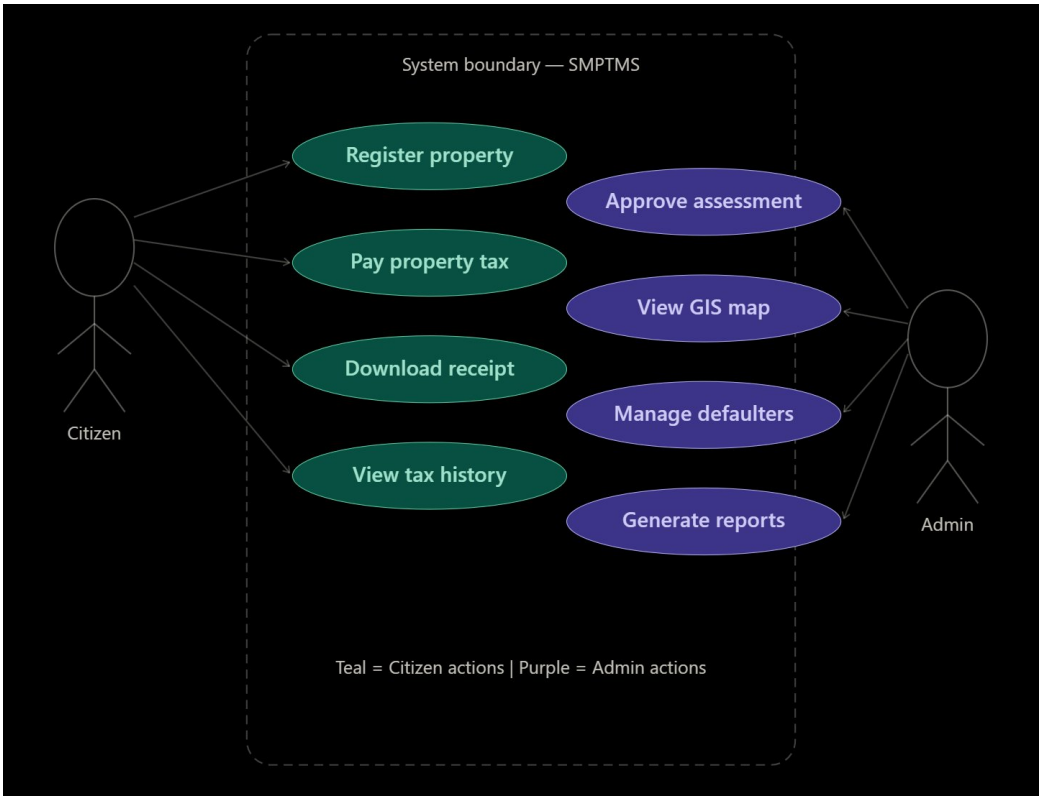


Figure 3.1: Use Case Diagram of SMPTMS

3.3.2 Activity Diagram

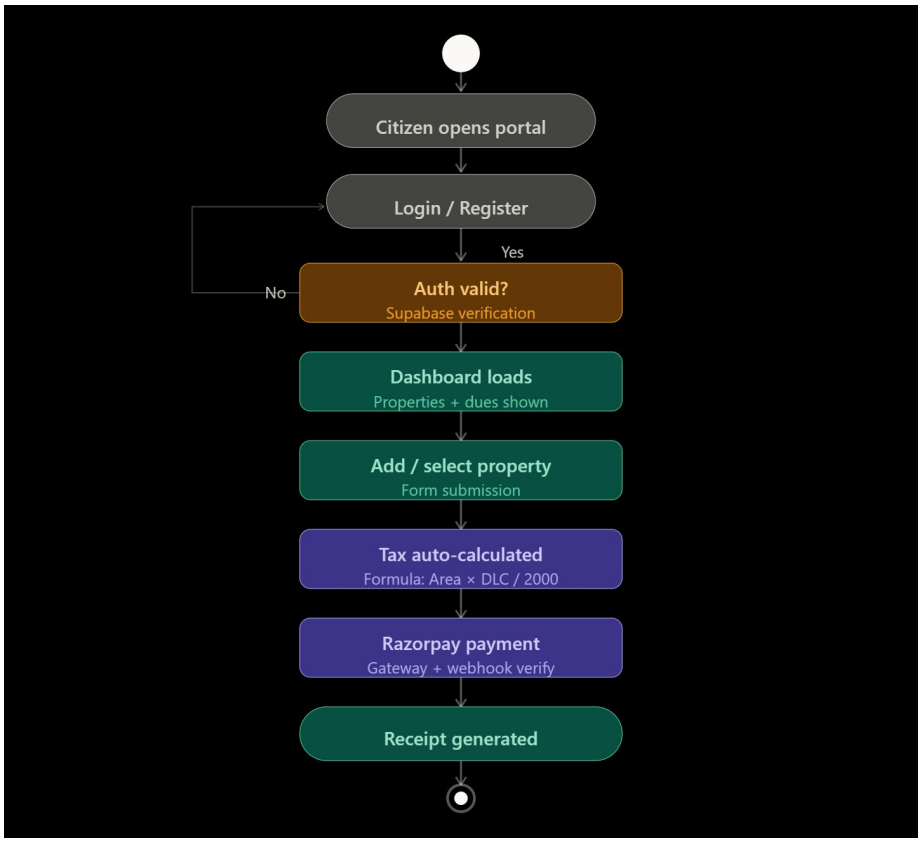


Figure 3.2: Activity Diagram – Citizen Tax Payment Flow

3.3.3 Data-Flow Diagram

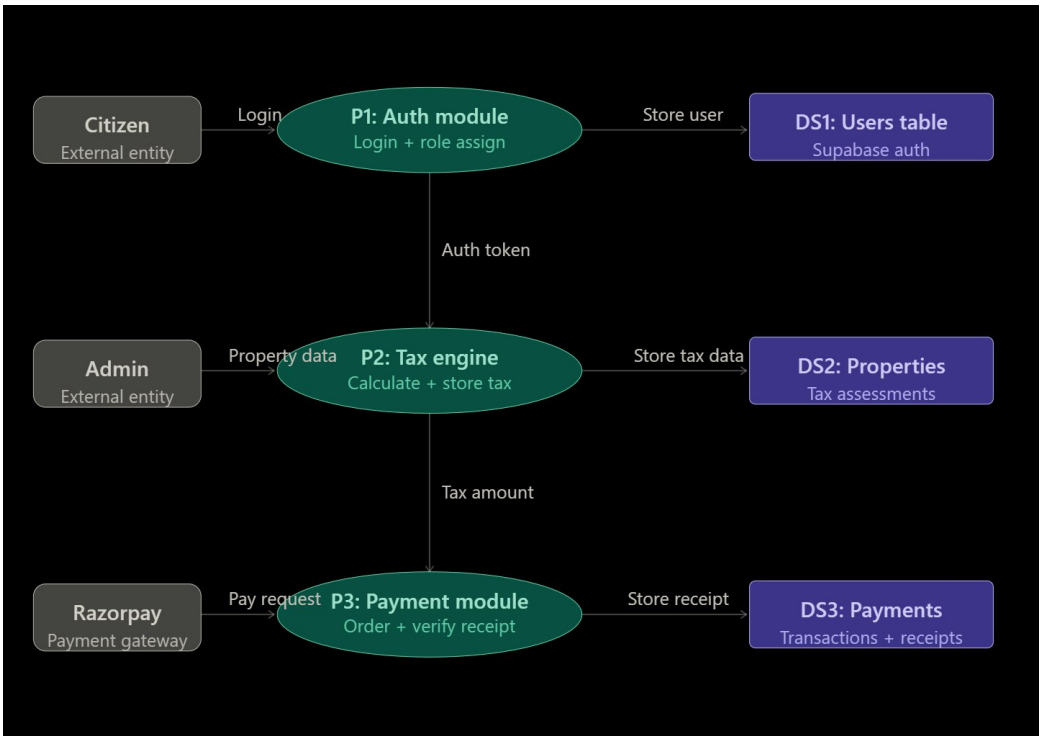


Figure 3.3: Data-Flow Diagram of SMPTMS

3.3.4 Class Diagram

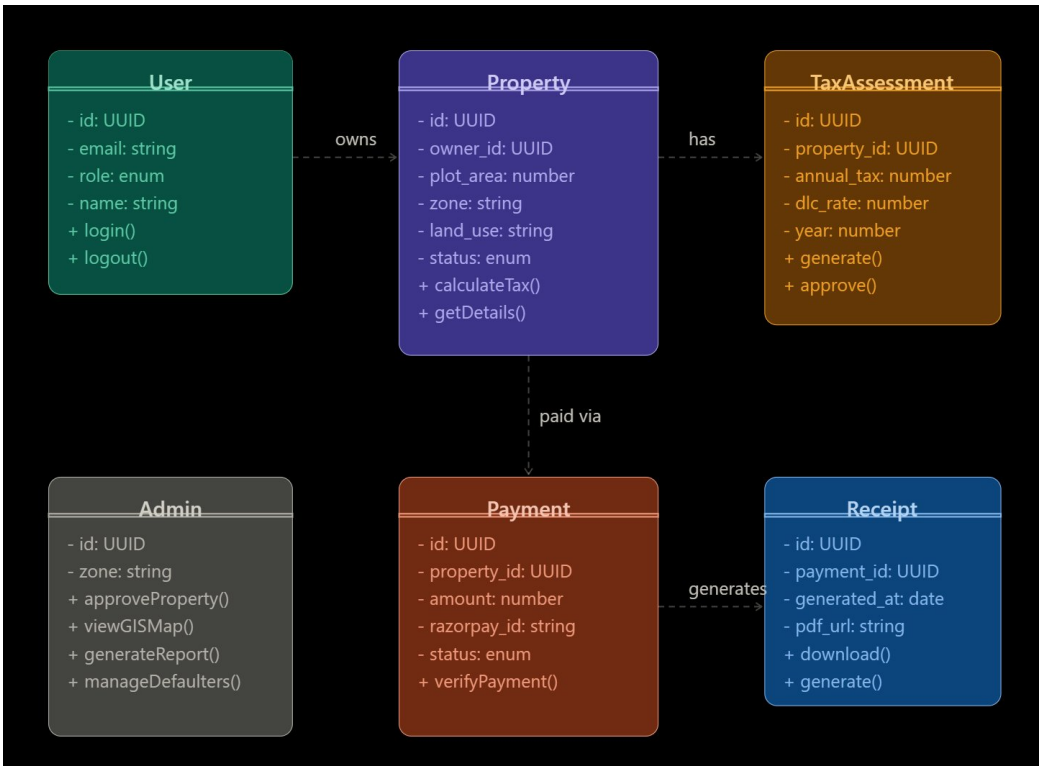


Figure 3.4: Class Diagram of SMPTMS

3.3.5 Sequence Diagram

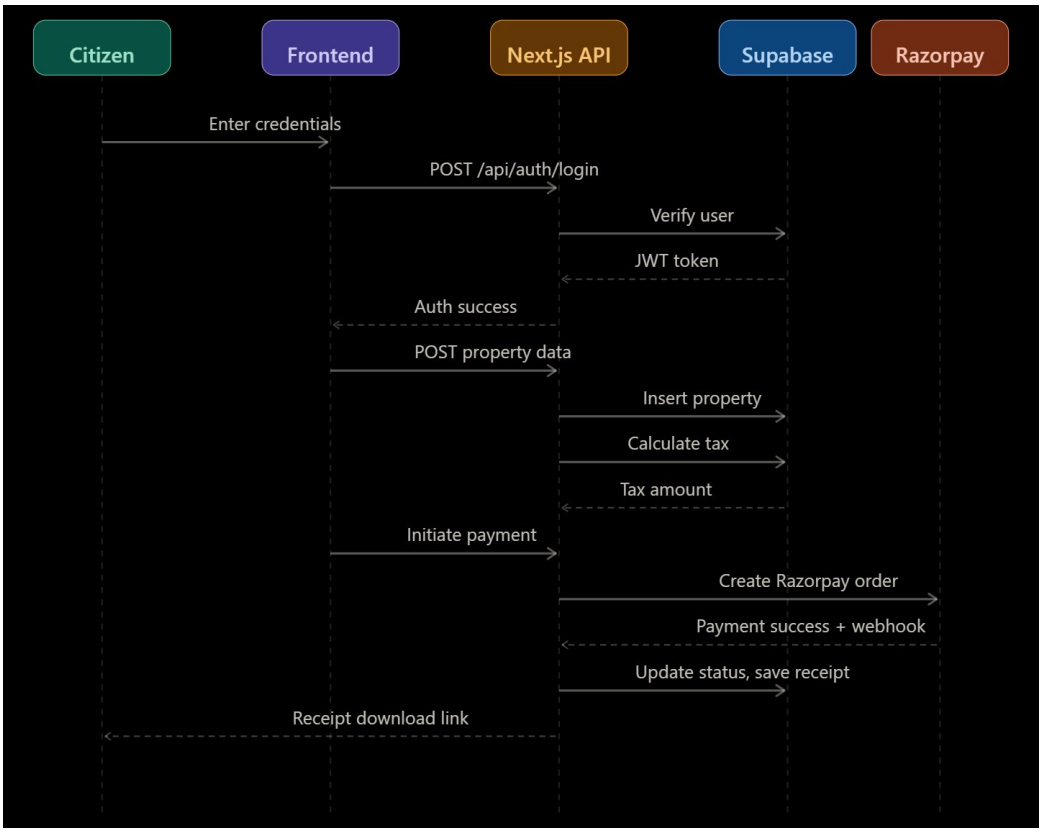


Figure 3.5: Sequence Diagram – Payment Processing Flow

Chapter 4
Methodology and Team

4.1 Introduction to Agile Methodology

Agile methodology breaks down project management into short, repeating cycles—these are called sprints. Instead of waiting until the very end to deliver a finished product, Agile teams roll out working pieces of the project bit by bit. This lets everyone gather feedback, make changes, and steadily improve things the whole way through.

With SMPTMS, we went with Agile because we had four modules being developed at the same time, each by a different team member. This setup needed a lot of coordination—people had to keep checking their work against each other's and make sure nothing was out of sync. Working in sprints allowed us to react quickly when testing showed something needed to change. We could adjust requirements and shuffle priorities at the end of each cycle instead of waiting for problems to pile up.

Here's what the Agile phases looked like for SMPTMS:

1. **Requirement Gathering:** The team started by digging into Jaipur Nagar Nigam's property tax challenges. We also looked at other city platforms like PCMC Pune and BBMP Bengaluru. This research helped us list both the must-haves and nice-to-haves, which went into the SRS (Software Requirements Specification). Things like GIS map integration, Razorpay payments, and role-based access all ended up at the top of our priorities.

2. **Design:** Next, we sketched out the system's overall structure—this included the database setup (using Supabase PostgreSQL), API routing with Next.js App Router, and various design diagrams like use case, activity, DFD, class, and

sequence. We locked in a three-tier architecture, which made it easier to split the work into four separate modules for parallel development.

3. **Development (Coding):** Each team member tackled their own module—and kept cycling through improvements. Khushal Jangid handled authentication and the user dashboard. Gaurav Pratap Singh built the tax calculation engine. Kapil Jarwal managed testing systems and set up the deployment pipeline. Himanshu Meena focused on GIS mapping and survey tools. We synced up every week to make sure all our parts worked nicely together.

4. **Testing:** Testing happened step by step—first with unit tests, then integration and system checks. We relied on Postman for API testing and also did manual browser testing to make sure our UI was smooth. You'll find all our test cases for each module in Chapter 5.

5. **Deployment:** Once everything was ready, we launched the application on Vercel's cloud. Continuous deployment was tied in with our GitHub repo, so updates went live fast. Supabase provided our managed PostgreSQL backend, and we set up Razorpay with both test and live payment credentials.

6. **Maintenance:** After deployment, the team kept an eye on Vercel logs and Supabase query performance to catch and fix any runtime bugs quickly. All issues found in final testing were tracked and patched up before we submitted the project report.

4.2 Team Members, Roles & Responsibilities

MEMBER 1: Khushal Jangid (22ESKCA057)		NAME OF MODULE: User Management	
NAME OF FUNCTIONALITY	**SOFT DEADLINE**	**HARD DEADLINE**	**DETAILS OF FUNCTIONALITY**
Setup & Grid	30/8/25	10/9/25	React.js frontend, Node.js backend setup, login using Next Auth
Styling	18/8/25	25/9/25	Design UI for user authentication and profile management
Integration (Auth)	20/9/25	10/10/25	Implement JWT authentication, user dashboard to view profile
User Dashboard	21/10/25	28/11/25	Create User APIs for registration, login, and data handling
Backend API	10/11/25	28/11/25	Develop CRUD APIs with tax API and error handling
Tech Integration	10/12/25	30/12/25	Integrate user details with better UX and error handling
Final Adjustment	30/1/26	20/1/26	Polish frontend with module testing and documentation
Documentation	28/2/26	20/2/26	Conduct User Module Testing and Calculation

Table 4.1: Member-1 Roles and Responsibilities

MEMBER 2: Gaurav Pratap Singh (22ESKCA035)		NAME OF MODULE: Tax Calculation	
NAME OF FUNCTIONALITY	**SOFT DEADLINE**	**HARD DEADLINE**	**DETAILS OF FUNCTIONALITY**
Schema Design	28/8/25	30/8/25	Design Supabase schema for tax records, property and income
Tax Data Entry Module	20/9/25	25/9/25	Create form for property entry and tax function
Tax Logic	27/10/25	30/10/25	Implement calculation logic: Tax = (Plot Area x DLC Rate) / 2000
API Development	25/11/25	30/11/25	Build API routes for entries and data sync
Integration (Supabase)	15/12/25	30/12/25	Add input validation and tax reports
Error Handling	10/1/26	15/1/26	Auto-generate tax reports of non-compliant entries
Report Generation	10/2/26	15/2/26	Integrate tax reports with frontend dashboard
Final Integration	23/2/26	15/3/26	Final testing of Tax Calculation module

Table 4.2: Member-2 Roles and Responsibilities

MEMBER 3: Kapil Jarwal (22ESKCA054)		NAME OF MODULE: Testing Module	
NAME OF FUNCTIONALITY	**SOFT DEADLINE**	**HARD DEADLINE**	**DETAILS OF FUNCTIONALITY**
Test Environment	28/8/25	30/8/25	Setup Postman and mock environment for API integration
Integration Testing	20/9/25	25/9/25	Design testing plan for APIs and automated test cases
Unit Testing Scripts	21/10/25	27/10/25	Create automated test cases for user and tax modules
API Testing	25/11/25	30/12/25	Testing all backend endpoints and integration points
Bug Report Sheet	25/11/25	15/1/26	Maintaining test results, deploy system on Vercel
Development Setup	15/1/26	15/2/26	Final test execution and environment validation
Final Testing	28/2/26	15/3/26	End-to-end system testing on live deployment
Documentation	–	–	Complete documentation of all test cases and results

Table 4.3: Member-3 Roles and Responsibilities

MEMBER 4: Himanshu Meena (22ESKCA047)		NAME OF MODULE: Surveying / GIS	
NAME OF FUNCTIONALITY	**SOFT DEADLINE**	**HARD DEADLINE**	**DETAILS OF FUNCTIONALITY**
Land Survey Form	2/8/25	12/8/25	Survey form: Parcel ID, area, coordinates
Map Integration	13/8/25	11/9/25	Map integration with polygon boundary and document upload
Survey Data Upload	1/9/25	10/10/25	Document upload, version history, timeline of survey changes
Survey History Tracking	1/10/25	15/11/25	Admin dashboard for surveyed vs pending properties
Parcel Status Dashboard	2/11/25	12/12/25	Duplicate parcel detection and geo-conflict handling
Duplicate Parcel Detection	1/1/26	10/1/26	PDF reports with ward/zone-wise date filters
Survey Report Generation	1/2/26	11/2/26	Auto alerts via email/SMS on survey updates
Final Integration	20/2/26	15/3/26	Final GIS module testing and documentation

Table 4.4: Member-4 Roles and Responsibilities

Chapter 5
System Testing

5.1 Introduction

Software testing is the systematic process of evaluating a system to verify that it behaves correctly and meets its specified requirements. For SMPTMS, testing was conducted across three levels — unit, integration, and full system — to validate every component before the platform was made available for live use. The primary testing tool was Postman for API-level testing, supplemented by manual browser-based testing for UI workflows and Razorpay's sandbox environment for payment validation.

5.2 Types of Testing Performed

5.2.1 Unit Testing

Each module was tested independently before integration. The tax calculation engine was tested by running a range of input combinations — varying plot sizes, zone categories, and DLC rates — through the formula and verifying computed outputs against manual calculations. The authentication module was tested by attempting valid and invalid login credentials, checking session expiry behaviour, and validating role switching between citizen and admin accounts. Each API endpoint was tested in isolation using Postman to confirm correct status codes, response payloads, and error handling.

5.2.2 Integration Testing

Integration testing focused on verifying correct data flow between modules. Key integration test scenarios included confirming that a property registered by a citizen appeared immediately on the admin GIS map with the correct zone and status, ver-

ifying that a completed Razorpay payment updated the property record in Supabase in real time via the webhook, and ensuring that the receipt generation function correctly pulled transaction details from the payment record. Cross-module interactions between the user management, tax calculation, and payment modules were validated in this phase.

5.2.3 System Testing

Full end-to-end testing was performed on the live Vercel deployment. Test scenarios covered the complete citizen workflow: account registration, property submission, tax calculation review, Razorpay sandbox payment completion, and receipt download. Admin workflows were also tested: logging in to the admin panel, viewing the GIS property map, filtering by zone and payment status, accessing the defaulter list, and generating zone-wise reports. All tests were repeated after bug fixes to confirm resolution.

5.3 Test Cases

NAME OF MODULE: User Management			TOOL: Postman + Manual	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC-UM-01	Valid user registration	Name: Kapil Jarwal, Email: kapil@gmail.com, Pass: Test@123	Account created, redirect to dashboard	Account created successfully
TC-UM-02	Login with correct credentials	Email: kapil@gmail.com, Pass: Test@123	Dashboard loads with user data	Dashboard loaded correctly
TC-UM-03	Login with wrong password	Email: kapil@gmail.com, Pass: wrong123	Error – Invalid credentials	Error displayed correctly
TC-UM-04	Login with unregistered email	Email: fake@test.com, Pass: Test@123	Error – User not found	Error displayed correctly
TC-UM-05	Access dashboard without login	Direct URL: /dashboard	Redirected to /login	Redirected correctly
TC-UM-06	Admin login with citizen credentials	Citizen credentials on admin panel	Access denied, role mismatch	Access correctly blocked
TC-UM-07	Session timeout check	User idle for extended period	Auto logout, redirect to login	Session expired correctly
TC-UM-08	Duplicate email registration	Same email registered twice	Error – Email already exists	Duplicate blocked correctly

Table 5.1: Test Cases – User Management Module

NAME OF MODULE: Tax Calculation			TOOL: Postman + Manual	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC-TC-01	Basic tax calculation – Zone C residential	Area: 280 sq.m, Zone C, Residential	Tax = (280 x DLC Rate) / 2000	Correct amount calculated
TC-TC-02	Tax calculation – Zone A commercial	Area: 520 sq.m, Zone A, Commercial	Higher tax due to Zone A DLC rate	Correct amount calculated
TC-TC-03	Minimum plot size	Area: 50 sq.m, Zone B, Residential	Low tax as per formula	Calculated correctly
TC-TC-04	Large commercial plot	Area: 2000 sq.m, Zone A, Commercial	Large tax verified manually	Matched manual calculation
TC-TC-05	Zero area input validation	Area: 0, Zone C	Error – Invalid plot area	Validation error shown
TC-TC-06	Negative area input	Area: -100	Error – Area cannot be negative	Validation blocked input
TC-TC-07	DLC rate change effect	Admin updates Zone B DLC rate	New rate applied to Zone B properties	Rate updated correctly
TC-TC-08	Multiple properties total dues	2 properties in dashboard	Correct total dues displayed	Correct total displayed

Table 5.2: Test Cases – Tax Calculation Module

NAME OF MODULE: Payment and Receipt			TOOL: Postman + Razorpay	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC-PM-01	Successful payment – test mode	Razorpay test card, Amount: Rs.1434	Payment success, status updated to Paid	Payment processed correctly
TC-PM-02	Payment cancelled by user	User cancels Razorpay popup midway	Payment status remains Pending	Status unchanged correctly
TC-PM-03	Invalid card details	Card: 0000 0000 0000 0000	Payment failed, Razorpay error	Failure handled correctly
TC-PM-04	Receipt generation after payment	Completed payment transaction ID	Downloadable PDF receipt generated	Receipt generated correctly
TC-PM-05	Payment webhook verification	Razorpay sends signed callback	Backend verifies signature, updates DB	Signature verified correctly
TC-PM-06	Double payment attempt	Pay again on already-paid property	Warning – Tax already paid	Blocked correctly
TC-PM-07	Receipt data accuracy	Download receipt after payment	Correct name, amount, date, property	All fields matched correctly
TC-PM-08	Admin real-time status update	Payment completed by citizen	Admin panel reflects paid status instantly	Real-time update confirmed

Table 5.3: Test Cases – Payment and Receipt Module

NAME OF MODULE: Surveying / GIS Map			TOOL: Manual + Leaflet.js	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC-GIS-01	Map loads with Jaipur coordinates	Open /dashboard-/map	Map centred on Jaipur, all markers visible	Map loaded correctly
TC-GIS-02	Paid property marker colour	Property with paid status	Green marker on map	Green marker displayed
TC-GIS-03	Pending property marker colour	Property with pending status	Orange marker on map	Orange marker displayed
TC-GIS-04	Search by property owner name	Search: Rajesh Kumar	Sidebar highlights property, map pans to location	Search worked correctly
TC-GIS-05	Filter pending properties only	Click Pending filter	Only pending properties shown	Filter applied correctly
TC-GIS-06	Filter paid properties only	Click Paid filter	Only paid properties shown	Filter applied correctly
TC-GIS-07	Property popup on marker click	Click any map marker	Popup shows owner, zone, tax, status	Popup displayed correctly
TC-GIS-08	Satellite view toggle	Click Satellite button	Map tiles switch to satellite imagery	Toggle worked correctly
TC-GIS-09	Total property count accuracy	21 properties in database	Header shows 21 Total	Count matched correctly
TC-GIS-10	Map responsive on mobile	Open map on mobile browser	Map renders correctly, controls accessible	Responsive confirmed

Table 5.4: Test Cases – GIS Map and Surveying Module

Chapter 6
Project Screenshots

6.1 Introduction

This chapter presents screenshots of the SMPTMS platform captured from the live deployment at **smptms.vercel.app**. The screenshots cover all major interfaces of the system — the public landing page, citizen authentication, property registration workflow, tax calculation, Razorpay payment gateway, receipt generation, and the complete admin control center including GIS mapping and analytics.

6.2 Public Landing Page

Figure 6.1: SMPTMS Homepage – Simple Property Tax, Transparent Management

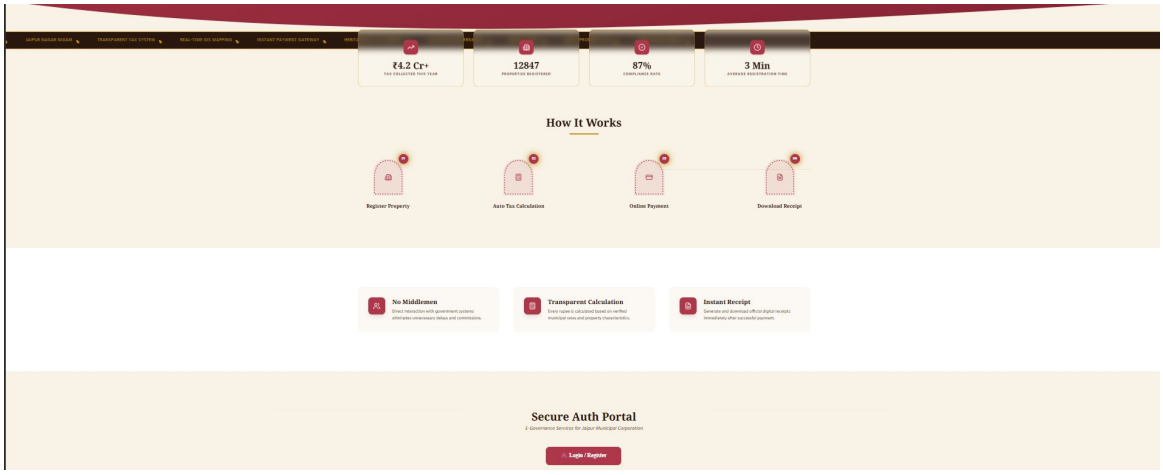


Figure 6.2: Landing Page – How It Works Section (Register, Calculate, Pay, Receipt)

6.3 Citizen Interface

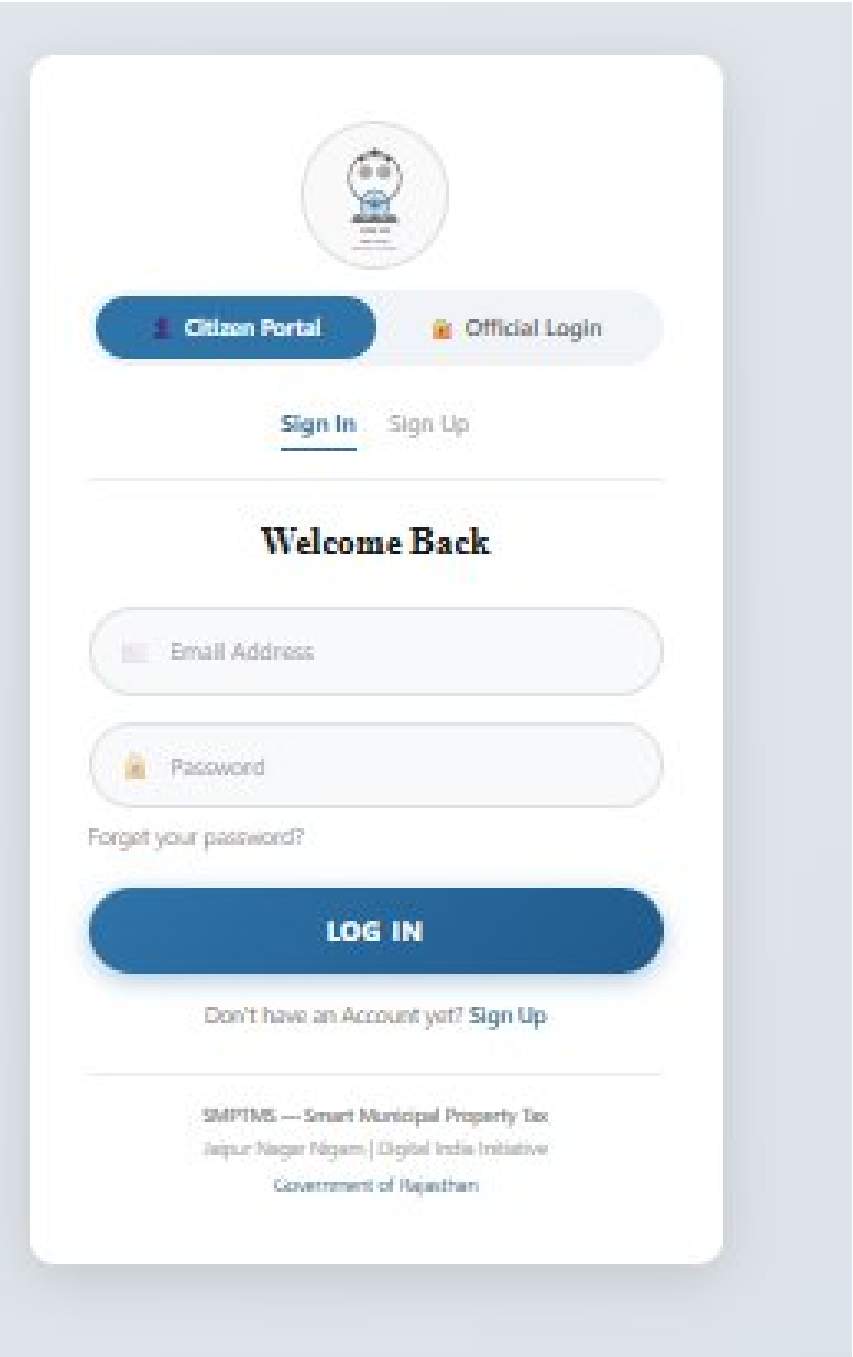


Figure 6.3: Citizen Login / Registration Page – Secure Auth Portal

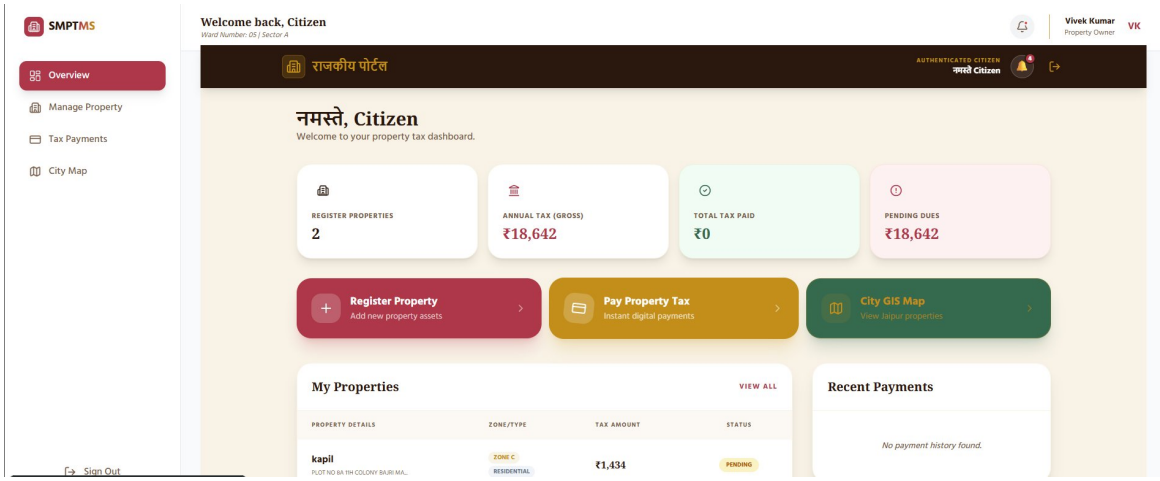


Figure 6.4: Citizen Dashboard – Property Portfolio, Dues, and Quick Actions

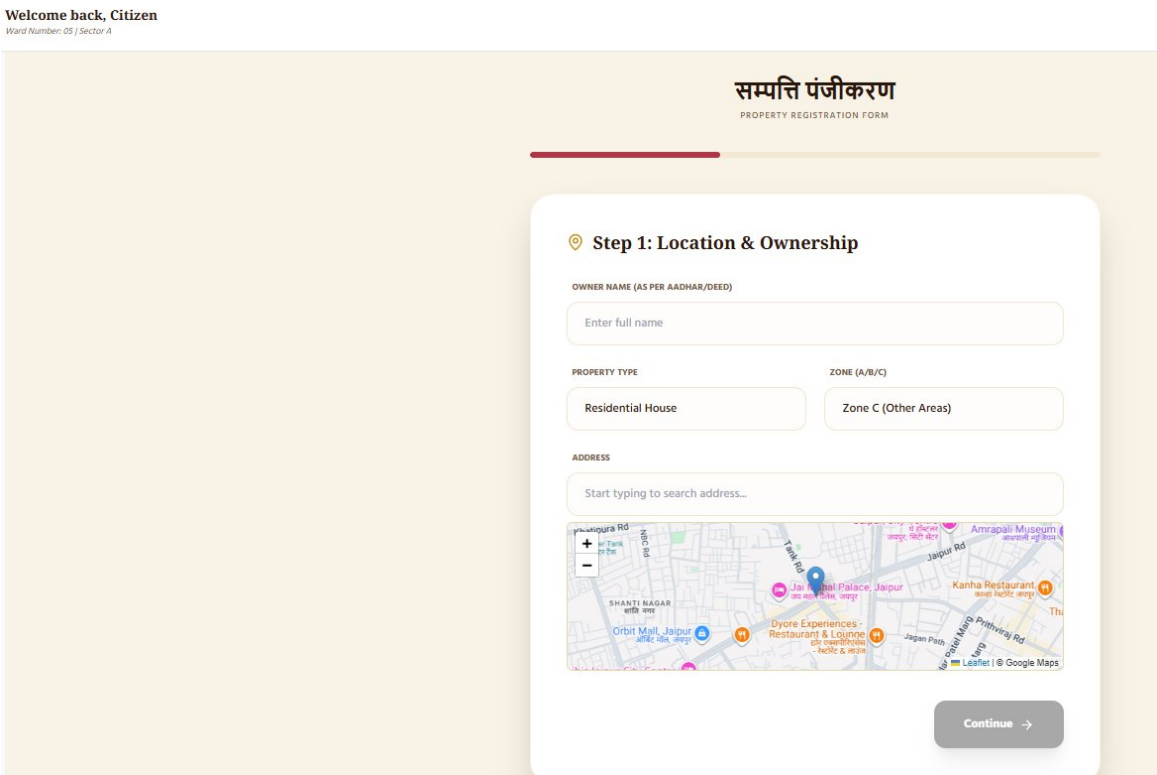


Figure 6.5: Property Registration – Step 1: Location & Ownership Details with Map Pin

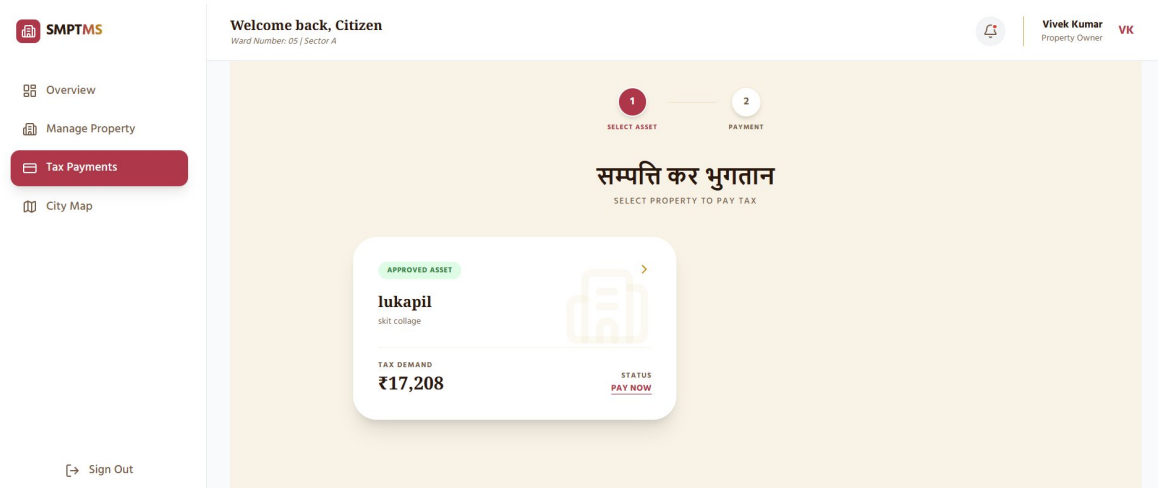


Figure 6.6: Property Registration – Step 2: Dimensions & Estimated Annual Tax Calculation

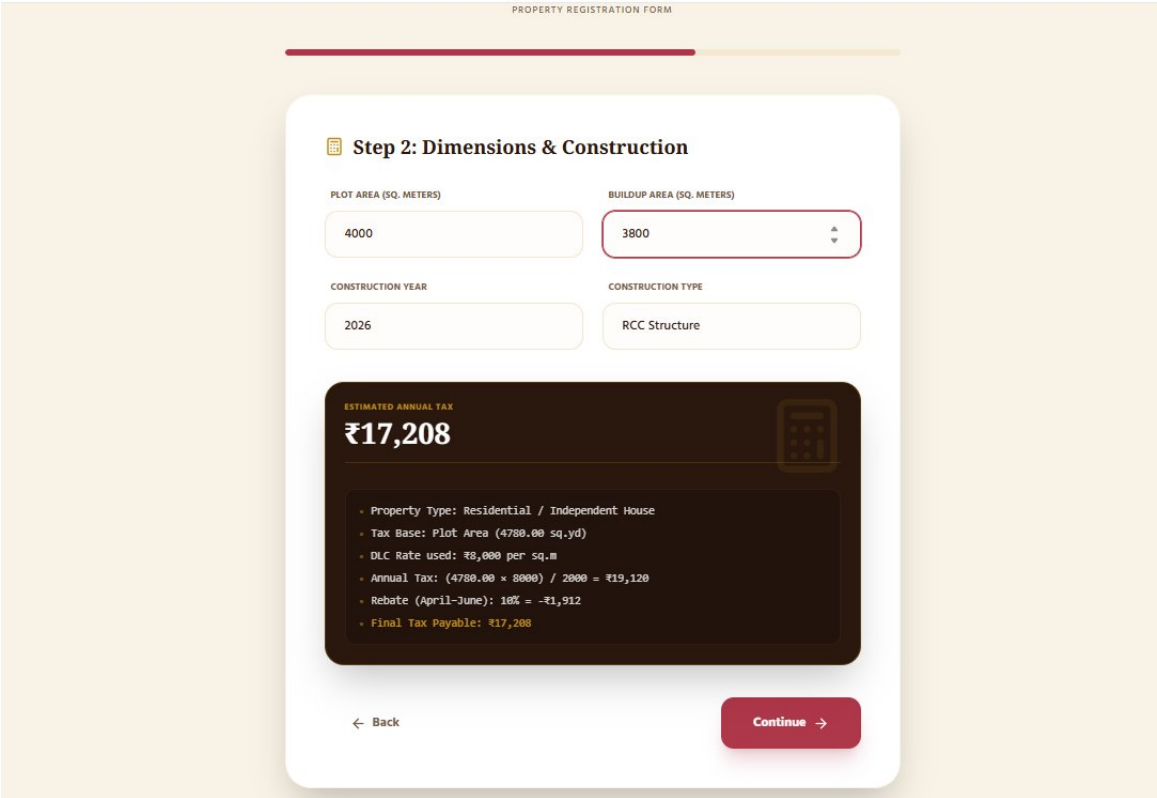


Figure 6.7: Tax Payments Page – Select Property to Pay Tax (Approved Asset View)

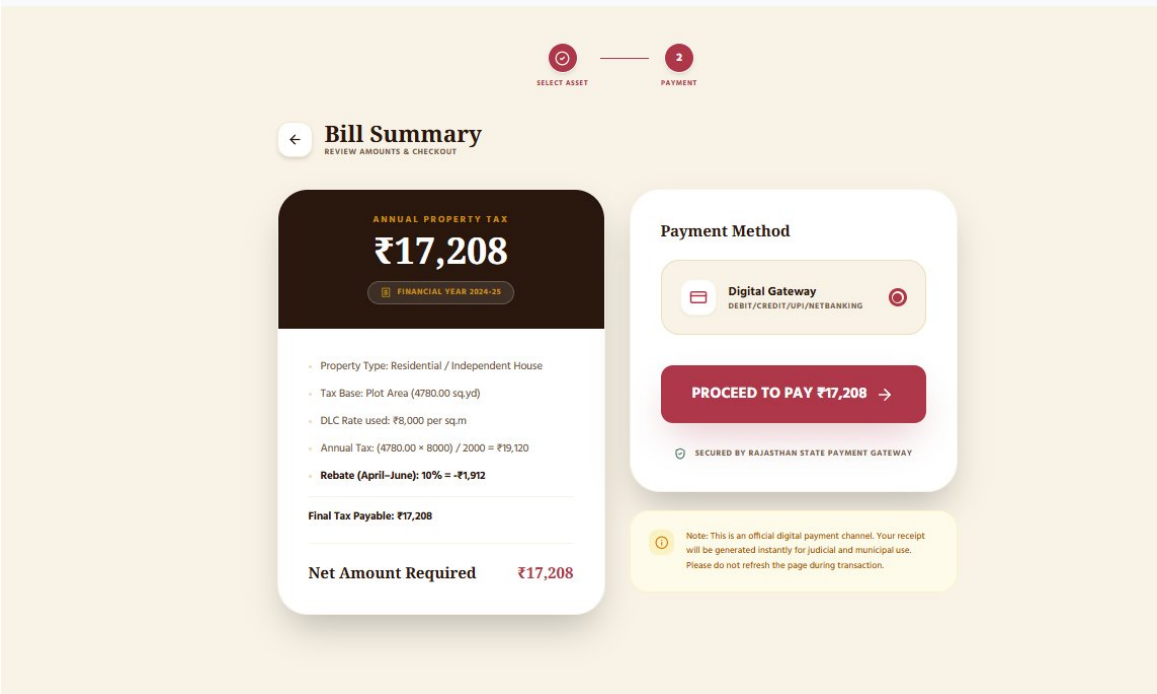


Figure 6.8: Bill Summary – DLC Rate Breakdown, Rebate Calculation, Final Tax Payable

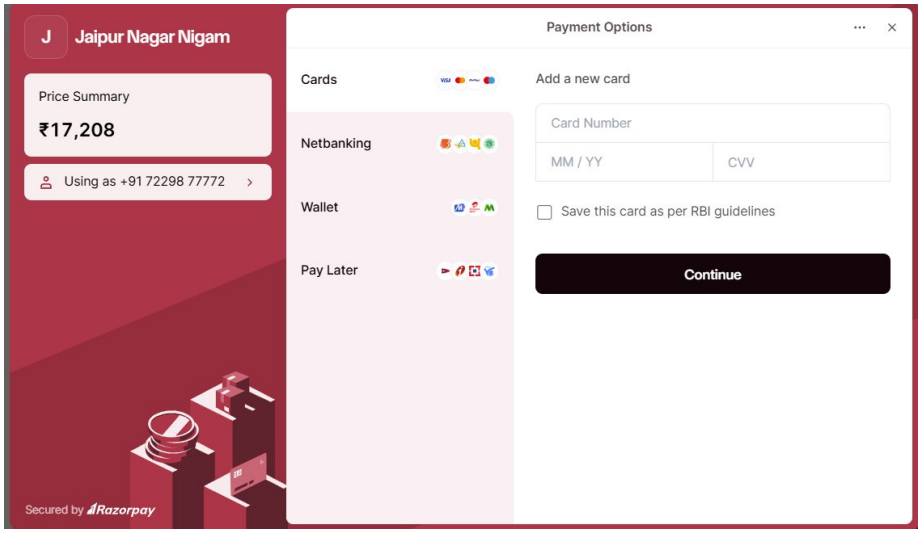


Figure 6.9: Razorpay Payment Gateway – Cards, Netbanking, UPI, Wallet Options

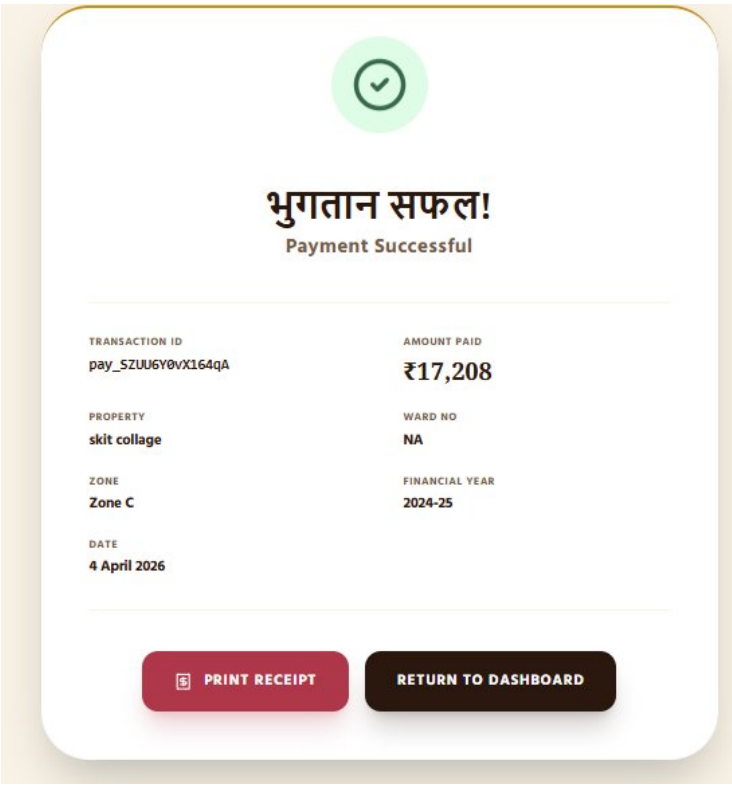


Figure 6.10: Payment Success Confirmation – Transaction ID, Amount, Print Receipt Button

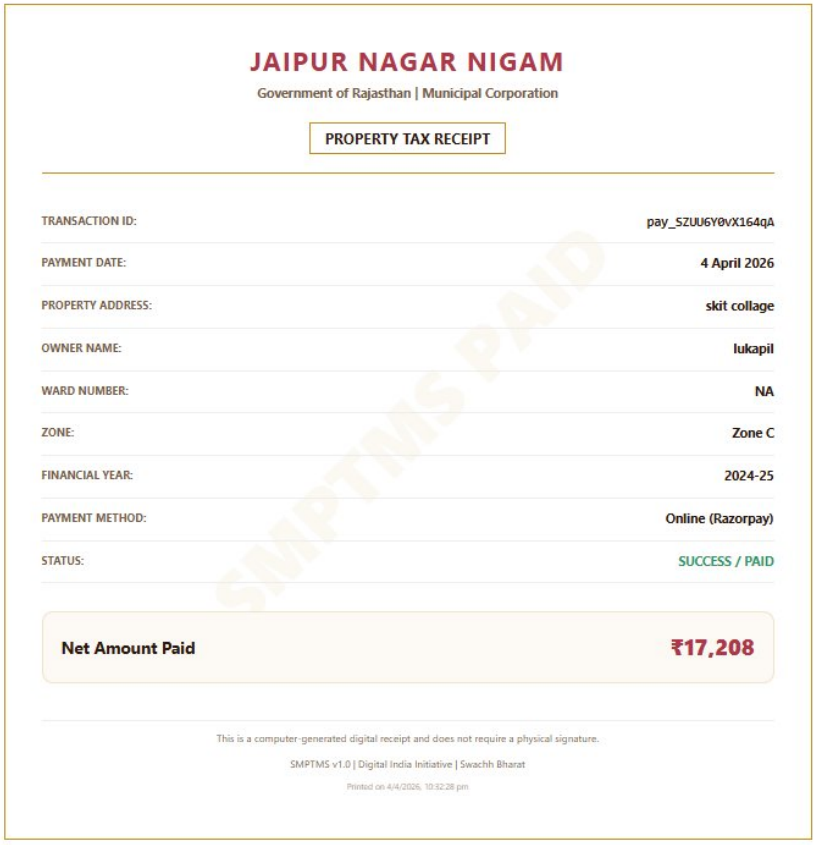


Figure 6.11: Official Property Tax Receipt – Jaipur Nagar Nigam, Razorpay Transaction Details

6.4 Admin Control Center

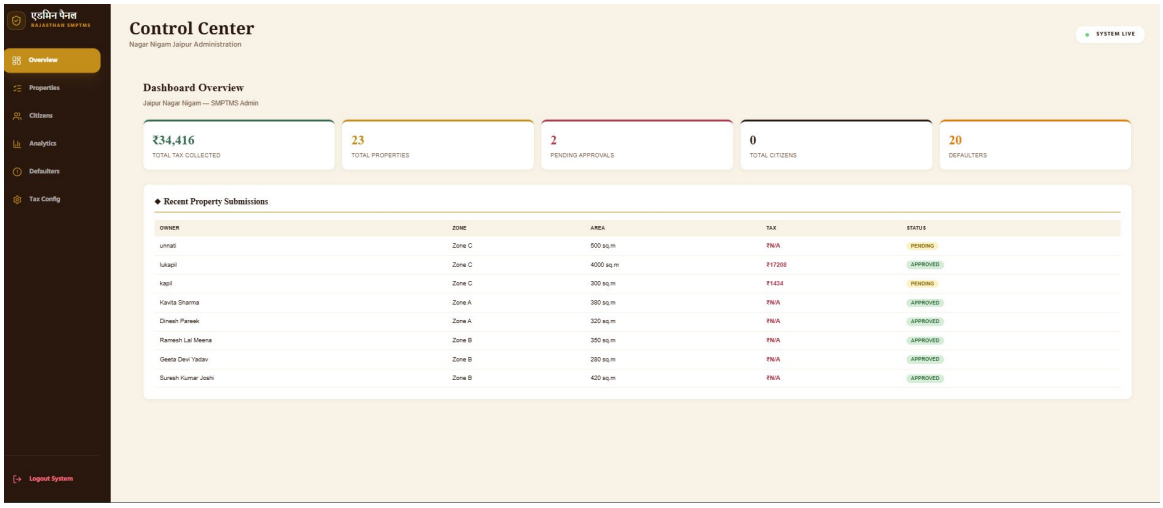


Figure 6.12: Admin Dashboard Overview – Total Tax Collected, Properties, Pending Approvals, Defaulters

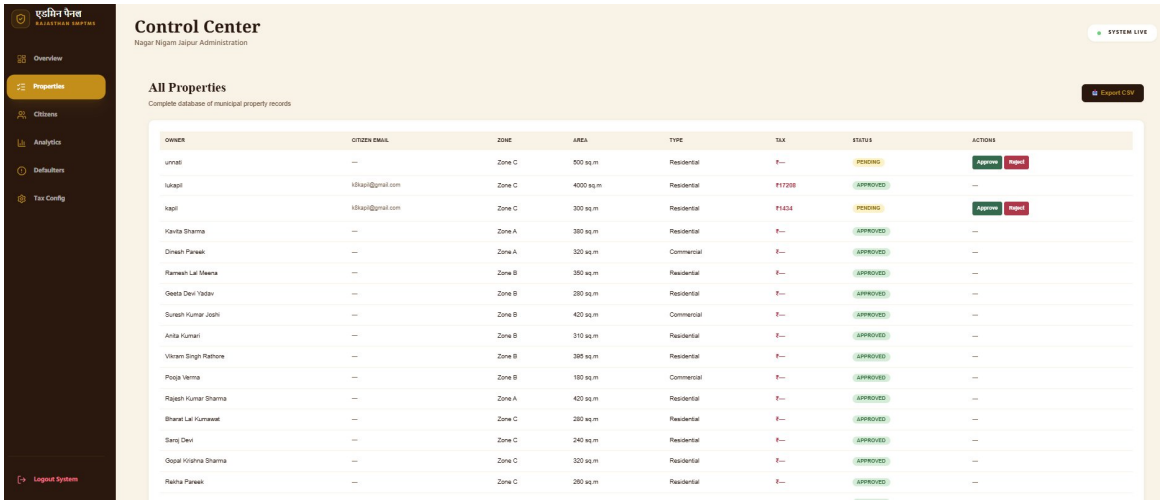


Figure 6.13: Admin – All Properties Database with Approve/Reject Actions

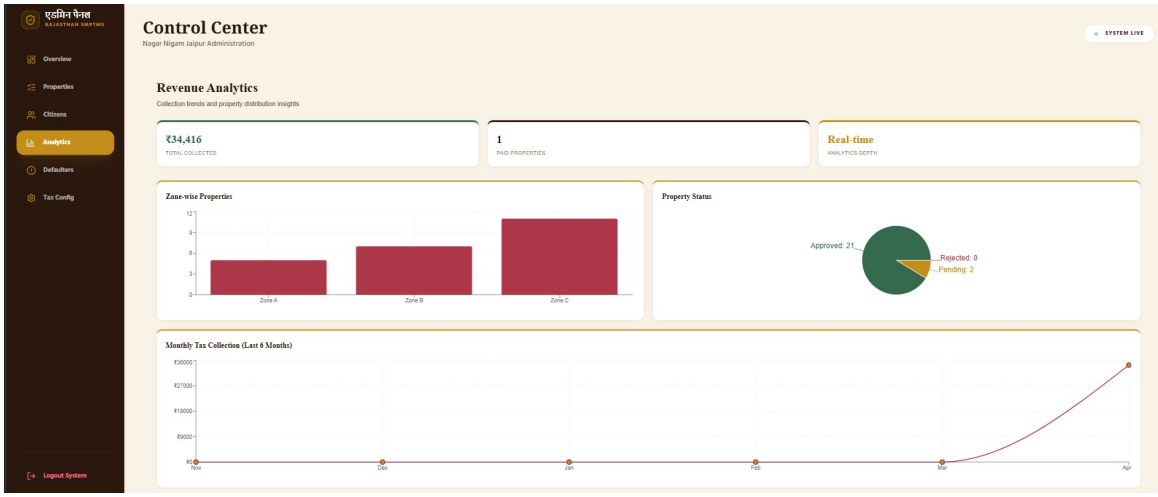


Figure 6.14: Admin Analytics – Zone-wise Bar Chart, Property Status Pie Chart, Monthly Collection Trend

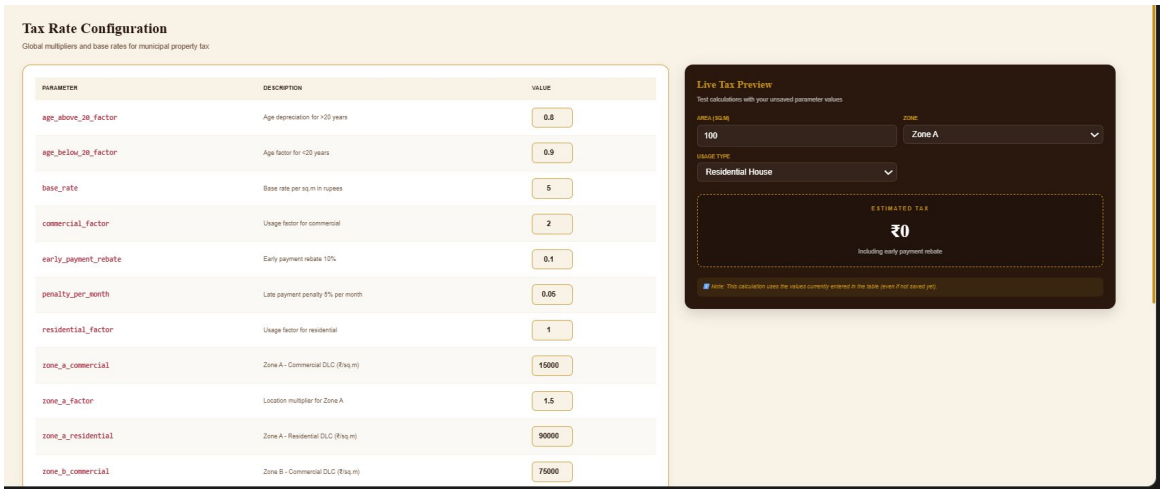


Figure 6.15: Admin Tax Rate Configuration – DLC Rates, Zone Factors, Live Tax Preview

6.5 GIS City Map

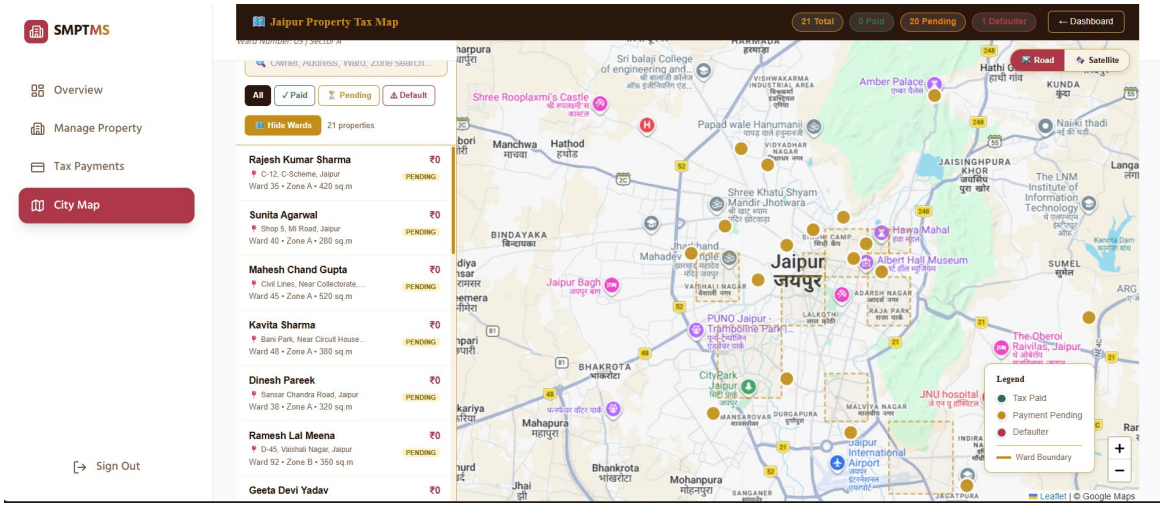


Figure 6.16: Jaipur Property Tax GIS Map – 21 Properties with Paid/Pending/Defaulter Markers and Ward Boundaries

Chapter 7
Conclusion and Future Scope

7.1 Conclusion

The Smart Property Tax Management System (SMPTMS) proves you don't need expensive gear or complicated tech setups to build a smooth, transparent property tax platform. By pulling together Next.js 14, Supabase, Leaflet.js, and Razorpay, the project tackles a real problem: messy municipal tax collection in hundreds of Indian cities.

In Jaipur, the platform brings the entire property tax process online. Now, anyone can register their property and pay taxes in just three minutes from any device with a browser. The tax calculation is fixed, open for everyone to see, and used across all city zones. That means there's no room for the old manual tricks that used to chip away at public trust in municipal tax systems.

Admins get a GIS-based property map showing, in real time, who's paying and who isn't. This helps them make smarter choices about enforcement and revenue planning. All four modules are up, running, and tested at smptms.vercel.app. The main goal? Cut out confusion, make it easy for people to pay, and rebuild trust in property tax collection for smaller Indian cities. SMPTMS does exactly that.

7.2 Future Scope

There's plenty of room to make SMPTMS even better:

- Link it up with Rajasthan's ULBHRMS platform so all city-level records sync automatically. This cuts down on duplicate entries and gives a big-picture look at state-wide revenue.

- Build a mobile app for iOS and Android with push notifications for upcoming dues, easy access to payment history, and offline viewing of property details.
- Use AI and satellite images to spot property anomalies in growing parts of Jaipur — catching undervalued or unregistered properties before they slip through the cracks.
- Add support for Hindi and Rajasthani dialects to help property owners in rural areas who aren't as tech-savvy.
- Integrate WhatsApp Business API for automated payment reminders and instant tax receipt PDFs, making it super easy for citizens to get what they need.
- Broaden the platform to include other municipal payments like trade licenses, water bills, and building plan approval fees — all from one portal.

Chapter 8
UN Sustainable Development Goals

8.1 Mapping with UN Sustainable Development Goals

Back in September 2015, 193 countries agreed on 17 Sustainable Development Goals (SDGs) at the UN General Assembly. These goals cover economic, social, and environmental challenges we all face. Every serious project, whether academic or professional, needs to show how it adds to at least one of these goals.

SMPTMS fits squarely under SDG 11, "Sustainable Cities and Communities," which pushes for cities that are inclusive, safe, resilient, and sustainable. One of the biggest factors in sustainable governance is giving municipalities financial independence. Property tax is the number one homegrown revenue stream for Urban Local Bodies in India. By digitizing and simplifying tax collection for Jaipur Nagar Nigam, SMPTMS strengthens city finances, makes governance more transparent, and improves urban services for everyone.

SDG Title: SDG 11 – Sustainable Cities and Communities		
S.No	**Objective**	**Outcome**
1	Improve municipal revenue collection through digitization of property tax administration	Increased tax compliance leading to better funding for civic infrastructure including roads, sanitation, and water supply in Jaipur
2	Make property tax assessment transparent and accessible to all citizens regardless of digital literacy level	Reduced scope for corruption and manual manipulation in tax calculations through a publicly visible, formula-based system
3	Enable online payment to remove physical barriers for citizens in remote wards who cannot easily visit municipal offices	Higher payment rates and reduced administrative burden on ward-level staff, enabling faster revenue realization
4	Provide GIS-based property mapping to support evidence-based urban planning and property record maintenance	Accurate, spatially-referenced property data enabling smarter infrastructure investment decisions by Jaipur Nagar Nigam

Table 8.1: SDG 11 Mapping for SMPTMS

References

[1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2014.

[3] Ministry of Housing and Urban Affairs, Government of India, "Municipal Finances in India: A Study of Property Tax," *National Institute of Urban Affairs Report*, New Delhi, India, 2021.

[4] S. Rajasekaran and G. A. Vijayalakshmi Pai, *Neural Networks, Fuzzy Logic, and Genetic Algorithms: Synthesis and Applications*, New Delhi, India: Prentice-Hall of India, 2010.

[5] Next.js Documentation, "App Router – Introduction," Vercel Inc., 2024. [Online]. Available: https://nextjs.org/docs

[6] Supabase Documentation, "Supabase Auth and PostgreSQL," Supabase Inc., 2024. [Online]. Available: https://supabase.com/docs

[7] Razorpay, "Razorpay Payment Integration Guide," Razorpay Software Pvt. Ltd., 2024. [Online]. Available: https://razorpay.com/docs

[8] Leaflet.js Documentation, "Leaflet – an open-source JavaScript library for mobile-friendly interactive maps," 2024. [Online]. Available: https://leafletjs.com/reference.html

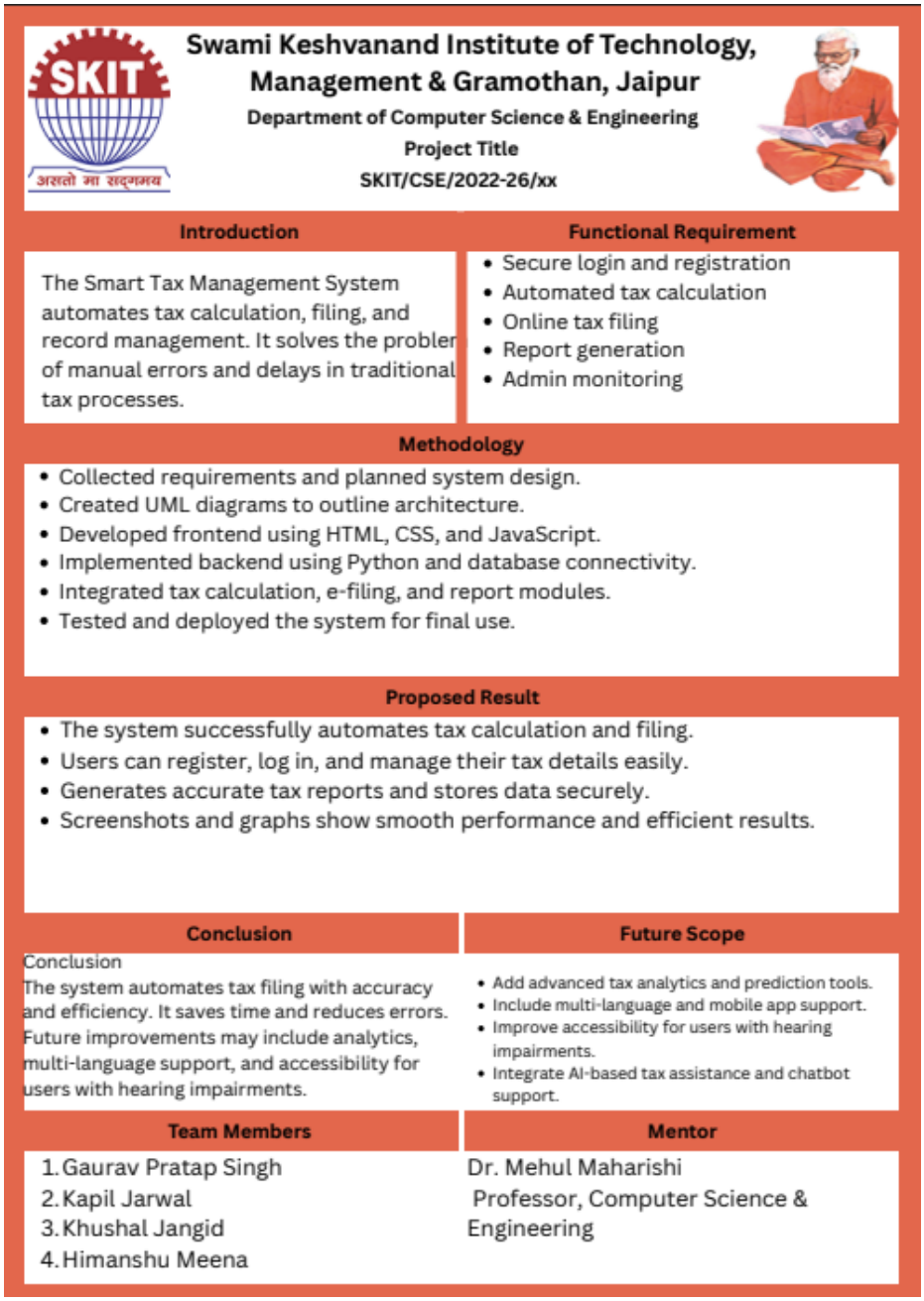
[9] Government of Rajasthan, "District Level Circle (DLC) Rates – Jaipur District," Stamps and Registration Department, Rajasthan, 2024.

[10] Ministry of Housing and Urban Affairs, "Smart Cities Mission – Digital Urban Governance Framework," Government of India, New Delhi, 2022.

[11] P. Ghosh and P. Bandyopadhyay, “Digitization of Property Tax in Urban Local Bodies: Evidence from Indian Cities,” *Journal of Urban Economics and Management*, vol. 8, no. 2, pp. 45–62, 2022.

[12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

Project Poster



Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science & Engineering

Project Title

SKIT/CSE/2022-26/xx

Introduction

The Smart Tax Management System automates tax calculation, filing, and record management. It solves the probler of manual errors and delays in traditional tax processes.

Functional Requirement

- Secure login and registration
- Automated tax calculation
- Online tax filing
- Report generation
- Admin monitoring

Methodology

- Collected requirements and planned system design.
- Created UML diagrams to outline architecture.
- Developed frontend using HTML, CSS, and JavaScript.
- Implemented backend using Python and database connectivity.
- Integrated tax calculation, e-filing, and report modules.
- Tested and deployed the system for final use.

Proposed Result

- The system successfully automates tax calculation and filing.
- Users can register, log in, and manage their tax details easily.
- Generates accurate tax reports and stores data securely.
- Screenshots and graphs show smooth performance and efficient results.

Conclusion

Conclusion
The system automates tax filing with accuracy and efficiency. It saves time and reduces errors. Future improvements may include analytics, multi-language support, and accessibility for users with hearing impairments.

Future Scope

- Add advanced tax analytics and prediction tools.
- Include multi-language and mobile app support.
- Improve accessibility for users with hearing impairments.
- Integrate AI-based tax assistance and chatbot support.

Team Members

1. Gaurav Pratap Singh
2. Kapil Jarwal
3. Khushal Jangid
4. Himanshu Meena

Mentor

Dr. Mehul Maharishi
Professor, Computer Science & Engineering

Figure 8.1: Project Poster

Certificate of Achievement

This section should contain certificates as proof of participation in any of the following activities undertaken by team members during the project period:

- Hackathon participation or winning certificate
- Research paper publication, presentation, or acceptance email
- Poster competition participation or award certificate
- Patent filing acknowledgement

(Attach relevant certificates here before final binding. If your research paper has been accepted but not yet published, include a screenshot of the acceptance email from the conference or journal.)

GitHub Link

Project Repository:

GitHub: `https://github.com/kapilk8/smptms`

Live Deployment:

Vercel: `https://smptms.vercel.app`

Project Details:

Project Name	Smart Property Tax Management System (SMPTMS)
Project ID	SKIT/2025-26/AI/Group-03
Tech Stack	Next.js 14, TypeScript, Supabase, Tailwind CSS, Leaflet.js, Razorpay
Deployment Platform	Vercel
Database	Supabase (PostgreSQL)
Payment Gateway	Razorpay
Session	2025 – 2026

Table 8.2: Project Technical Summary